Ders 13 Genişletilmiş Notu: İkili Arama Ağacında Silme, Heap Ağacı ve Heap Sort

Ders 13 Genişletilmiş Notu: İkili Arama Ağacında Silme, Heap Ağacı ve Heap Sort

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. İkili Arama Ağacı Tekrarı
- 2. İkili Arama Ağacına Ekleme ve Özyinelemeli Ekleme Kodu
- 3. İkili Arama Ağacından Silme
- 4. Heap Ağacı ve Üç Şartı
- 5. Heap'in Dizide Saklanması
- 6. Heap'e Ekleme ve Sift Up
- 7. Heap Sort
- 8. Sift Down ve findBigChild Fonksiyonu
- 9. Heap Sort Kodunun Tam İzlemesi
- Örnek Sorular
- Dersin Toparlaması

1. İkili Arama Ağacı Tekrarı

Ders tamamen online yapıldı. Hoca, geçen hafta önce genel ağaçların, sonra ikili ağaçların (binary tree), en son da ikili arama ağaçlarının (binary search tree, BST) anlatıldığını hatırlatarak başladı.

İkili arama ağacının mantığı şudur: kökün solunda kökten küçük değerler, sağında kökten büyük değerler yer alır. Bu kural yalnız kök için değil, her alt ağaç için de geçerlidir. Hocanın ekrandaki örneğinde 8'in solunda 8'den küçükler, sağında büyükler; 17'nin solunda 17'den küçükler vardı.

Ağaç yaklaşık olarak dengeliyse arama ve ekleme işlemleri $O(\log n)$ karmaşıklıkta yapılır. Bunun nedeni, her karşılaştırmada kalan bilginin yaklaşık yarısının elenmesidir. Hocanın örneğinde 9 aranırken önce kök 15 ile karşılaştırılır. 9 < 15 olduğu için ağacın sağ yarısı tek seferde elenir. Sonra 9 > 8 olduğu için 8'in solundaki bütün değerler elenir. 9 üçüncü adımda bulunur.

Ağacın tam dengeli olması gerekmez; yaklaşık dengeli olması yeterlidir. Ancak sürekli ekleme ve silme yapılırken denge bozulur ve bu denge yeniden düzeltilmezse, bir süre sonra ağacın bütün elemanları tek bir yöne yığılabilir. Bu durumda ağaç bir bağlı listeye döner ve BST kullanmanın bir avantajı kalmaz: işlemler $O(n)$ olur.

> **Hocanın vurgusu:** Dengeli bir BST'de arama, ekleme ve silme $O(\log n)$ sürer. Ağaç tek tarafa yığılırsa bu avantaj kaybolur.

2. İkili Arama Ağacına Ekleme ve Özyinelemeli Ekleme Kodu

2.1 Ekleme işleminin hatırlatılması

BST'ye ekleme (insert) yapılırken ağaçta sürekli yer değiştirme işlemleriyle uğraşmamak için yeni eleman her zaman bulunan **en uygun yaprağa** yerleştirilir. Hocanın örnek ağacının kökü 22'dir.

14'ün eklenmesi:

- 14 < 22, sola gidilir (11).
- 14 > 11, sağa gidilir (13).
- 14 > 13. 13 bir yapraktır, yani çocukları `NULL`'dır. 14, 13'ün sağına yerleştirilir.

24'ün eklenmesi:

- 24 > 22, sağa gidilir (25).
- 24 < 25, sola gidilir. 25'in solu `NULL` olduğu için 24, 25'in soluna yerleştirilir.

Kısaca ekleme işleminde eleman olabilecek en alt seviyeye, bir yaprağın altına yerleştirilir.

2.2 Bağlı liste ile saklama

Hoca daha önce konuşulan bir noktayı tekrarladı: ağaçta hareket varsa, yani sürekli ekleme ve çıkarma yapılıyorsa ikili ağacı bağlı liste (linked list) yapısında saklamak doğru olandır. Ağaç çok boşsa dizide saklamak çok sayıda gözü boşuna ayırmak demektir. Ayrıca sürekli ekleme ve silmelerde ağacın yapısı bozulacağı için bağlı liste yapısı işi her zaman kolaylaştırır.

> **Hocanın vurgusu:** Sürekli ekleme ve silme yapılan ağaçlar bağlı liste yapısında saklanmalıdır.

Her düğüm bir değer (`value`), sol alt düğümün adresini tutan bir `left` pointer'ı ve sağ alt düğümün adresini tutan bir `right` pointer'ı içerir. Yaprak düğümlerin hem solu hem sağı `NULL`'dır.

2.3 Özyinelemeli ekleme

Hoca klasik (döngülü) eklemeyi geçen hafta anlattığı için bu derste özyinelemeli (recursive) sürümü gösterdi. Bunun yalnızca notlar arasında bir örnek bulunsun diye anlatıldığını, klasik yapılmasının hiçbir sakıncası olmadığını, hatta kendisinin klasik yöntemi tercih ettiğini söyledi. Özyinelemeli sürümün belirgin bir avantajı yoktur.

Önce yeni düğüm için bellekte yer ayıran bir `newNode` fonksiyonu yazılır. Fonksiyonun dönüş tipi düğüm tipinde bir pointer'dır, çünkü ayrılan yerin adresi döndürülecektir. Bellek `malloc` ile ayrılır. Hoca, ayrılan pointer'ın `NULL` olup olmadığının mutlaka kontrol edilmesi gerektiğini, yer ayrılamadıysa işlemlere devam edilmemesi, gerekirse hata verilip programdan çıkılması gerektiğini söyledi; bunun ayrıntısını programlama derslerinde görülen dinamik bellek konusuna bıraktı. Yeni düğümün değeri yazılır, sol ve sağ pointer'ları `NULL` yapılır, çünkü yeni düğüm bir yaprak olarak yerleşir.

`insert` fonksiyonu kökü ve eklenecek değeri alır:

- Kök `NULL` ise ağaç (ya da o alt ağaç) boştur. Yeni düğüm oluşturulur ve adresi döndürülür.
- Eklenecek değer kökün değerinden küçükse sol alt ağaca özyinelemeli olarak eklenir.
- Büyükse sağ alt ağaca eklenir.
- Kök adresi geri döndürülür.

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int value;
    struct node *left;
    struct node *right;
};

struct node *newNode(int value) {
```

```
    struct node *temp = malloc(sizeof(struct node));
    if (temp == NULL) {               /* yer ayrılamadı */
        printf("bellek ayrilamadi\n");
        exit(1);
    }
    temp->value = value;
    temp->left = temp->right = NULL;  /* yeni düğüm yapraktır */
    return temp;
}

struct node *insert(struct node *root, int value) {
    if (root == NULL)                 /* ağaç (ya da alt ağaç) boş */
        return newNode(value);
    if (value < root->value)
        root->left = insert(root->left, value);
    else
        root->right = insert(root->right, value);
    return root;
}
```

Ana programda başlangıçta boş bir ağaç için önce kök oluşturulur: 15 kökün değeri olur. Hocanın örneğinde 15 bellekte 2000 numaralı adreste duruyorsa `root` artık 2000'i gösterir. Ardından `insert(root, 17)`, `insert(root, 11)` gibi çağrılarla ekleme yapılır. Hoca kodu yazıp adım adım izlemeyi öğrencilere bıraktı.

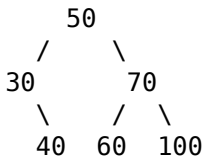
Kod `gcc -Wall` ile derlenip 15, 17, 11, 9, 13 sırasıyla eklendiğinde ağacın inorder dolaşımı `9 11 13 15 17` verdi; yani ekleme BST kuralını koruyor.

3. İkili Arama Ağacından Silme

Hoca, bağlı liste, ağaç, graf gibi bütün veri yapıları üzerinde yapılan temel işlemlerin üç tane olduğunu söyledi: yeni eleman ekleme (insert), arama (search) ve silme (delete). Bunlara ek olarak bir elemanın değiştirilmesi de olabilir. Ekleme ve arama geçen hafta görülmüştü. Bu derste silme görüldü.

Eklemede eleman hep en uygun yaprağa konuyordu. Silmede durum farklıdır, çünkü silinecek eleman ağacın herhangi bir yerinde olabilir. Bu yüzden elemanın nerede olduğuna göre üç durum ayrı ayrı ele alınır.

Hocanın örnek ağacı:

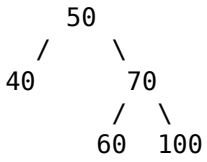


3.1 Durum 1: Silinecek eleman yaprakta

Örneğin 60 silinsin. Yapılacak özel bir iş yoktur: düğüm silinir ve onu gösteren pointer, yani 70'in sol pointer'ı `NULL` yapılır. Bu en kolay durumdur.

3.2 Durum 2: Silinecek düğümün tek çocuğu var

Örneğin 30 silinsin. 30'un tek çocuğu 40'tır. 40, 30'un yerine geçer:



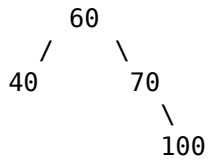
Çocuğun silinen düğümün solunda ya da sağında olması fark etmez. Silinen düğüm ebeveyninin solundaysa, onun bütün alt ağacı da ebeveynden küçüktür. Dolayısıyla tek çocuk yukarı çıktığında BST kuralı bozulmaz.

Hoca bunu ikinci bir örnekle gösterdi: 50'nin solunda 30, 30'un solunda 20 olsun (30'un sağı `NULL`), sağ tarafta yine 70, 60, 100 olsun. 30 silinirse 20 kolayca 30'un yerine çıkar. 50 artık 30'u değil 20'yi gösterir ve 30 için ayrılan bellek serbest bırakılır (`free`). 20 de 50'nin solunda kalan, 50'den küçük bir değer olduğu için hiçbir şey bozulmaz.

3.3 Durum 3: Silinecek düğümün iki çocuğu var

Aynı ağaçta (50; 40, 70; 60, 100) bu kez kök 50 silinsin. Yapılması gereken, **sağ alt ağacın en küçük değerini bulup yukarı almaktır**.

- 50'nin sağ alt ağacı 70, 60 ve 100'den oluşur.
- Bu alt ağacın en küçük değeri 60'tır.
- 60, 50'nin yerine geçer.



Hocanın gerekçesi, en az hareketle silmeyi tamamlamaktı. Sağ alt ağacın en küçük elemanı, sol alt ağaçtaki her elemandan büyük, sağ alt ağaçtaki diğer her elemandan küçüktür. Bu yüzden silinen düğümün yerine konduğunda BST kuralı korunur.

Hoca daha zor bir örnek olsun diye ağacı sağa doğru büyüterek 200'ü sildi. 200'ün sağ alt ağacının en küçük elemanı 220'ydi ve 220 yukarı alındı. 220'nin solu boş kaldı, sağında 250 vardı ve ağacın düzeni bozulmadı.

Son olarak hoca kökün silinmesini bir örnekle daha gösterdi: kök 100 silindiğinde sağ alt ağacın en küçüğü olan 120 köke çıkar, sol tarafta 70 ile başlayan alt ağaç olduğu gibi kalır. Hoca, gerçekte ağaç bağlı liste yapısında saklandığı için bu işlemin pointer'lar üzerinde yapılacağını hatırlattı.

> **Hocanın vurgusu:** Ertesi gün verilecek ödevde ağaçtan silme yapılması istenecek. Bu yüzden hoca silme algoritmasının kodunu yazmadı.
>
> **Ders sonrası not:** Hoca, sağ alt ağacın en küçük elemanının "yaprakta olacağını" söyledi. Doğrusu: Bu elemanın sol çocuğu yoktur ama sağ çocuğu olabilir. Sağ çocuğu varsa, eleman yukarı alındıktan sonra sağ çocuğu onun eski yerine bağlanır; bu, Durum 2'deki tek çocuklu silmedir.
>
> **Ek bilgi (derste anlatılmadı):** Sağ alt ağacın en küçük elemanına inorder ardıl (inorder successor) denir. Aynı iş sol alt ağacın en büyük elemanı (inorder predecessor) ile de yapılabilir; bu eleman da sol alt ağaçtaki her şeyden büyük, sağ alt ağaçtaki her şeyden küçüktür.

4. Heap Ağacı ve Üç Şartı

Hoca yeni bir ağaç türüne geçti: heap ağacı (heap tree). Bu kavram, dinamik bellek ayırmada (dynamic allocation) sözü edilen bellekteki heap bölgesiyle aynı şey değildir. Buradaki heap bir ağaç türüdür.

> **Hocanın vurgusu:** Heap ağacı, bellekteki heap bölgesinden farklı bir kavramdır.

Heap ağacının kullanım alanları:

- **Heap sort:** $O(n \log n)$ karmaşıklıkla sıralama.
- **Öncelikli kuyruklar (priority queue):** İlerleyen haftalarda görülecek; heap ağacı kullanır.
- **Bir dizinin k. büyük elemanına erişme:** Örneğin büyüklükçe üçüncü ya da beşinci sıradaki elemanı bulmak.

Bir ağacın heap olması için üç özelliği sağlaması gerekir:

1. Dengeli (balanced) olmalıdır.
2. Sola dayalı (left justified) olmalıdır.
3. Hiçbir düğüm ebeveyninden (parent) büyük olmamalıdır (max heap) ya da hiçbir düğüm ebeveyninden küçük olmamalıdır (min heap).

4.1 Birinci şart: Dengeli olma

Bu tanım geçen hafta görülmüştü. En alt seviye n. seviye, onun üstü $n-1$. seviye, onun üstü $n-2$. seviye olsun; kök 0. seviyededir. Bir ağacın dengeli olması için 0. seviyeden $n-2$. seviyeye kadar olan **bütün düğümlerin iki çocuğu olmalıdır**.

Hoca iki karşı örnek çizdi. Birincisinde $n-2$. seviyedeki bir düğümün çocuğu eksikti; bu ağaç dengeli değildir. İkincisinde de üst seviyelerde çocuğu olmayan bir düğüm vardı; o da dengeli değildir.

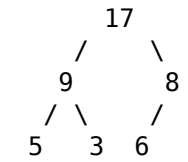
4.2 İkinci şart: Sola dayalı olma

Sola dayalı olma, son seviyedeki yaprak düğümlerin soldan başlayarak, arada boşluk bırakmadan dizilmesi demektir. Hoca bir sola dayalı ve bir sola dayalı olmayan ağaç çizdi. Ardından hem dengeli hem sola dayalı bir ağaç gösterdi: $n-2$. seviyeye kadar bütün düğümlerin ikişer çocuğu vardı ve son seviyedeki yapraklar sola yaslanmıştı.

4.3 Üçüncü şart: Max heap ya da min heap

Üçüncü şart düğümlerin ebeveynlerine göre durumudur. İhtiyaca göre iki türlü olabilir.

Max heap: Hiçbir düğüm ebeveyninden büyük olamaz. Hocanın örneği:



Burada 9 < 17, 8 < 17, 5 < 9, 3 < 9, 6 < 8'dir. Çocukların hepsi ebeveynlerinden küçüktür. Bu ağaç bir BST değildir: 9'un solda, 8'in sağda olmasının ya da 5'in solda, 3'ün sağda olmasının heap için hiçbir önemi yoktur. Önemli olan yalnız ebeveyn ile çocuk arasındaki ilişkidir.

Min heap: Hiçbir düğüm ebeveyninden küçük olamaz; yani ebeveynler çocuklardan büyük olamaz. Hocanın ikinci örneğinde kök 5'tir ve şu ilişkiler vardır: 7 > 5, 13 > 5, 14 > 13, 18 > 13, 9 > 7, 12 > 7, 23 > 12.

Hoca ilk başta 13'ün altına 11 yazmıştı. Bir öğrenci 11 < 13 olduğu için min heap şartının sağlanmadığını fark etti ve değer 14 olarak düzeltildi.

Soru: Min heap örneğinde 13'ün altına 11 yazıldı; orası şartı sağlamıyor gibi. **Cevap:** Evet, doğru; 11 yerine 14 yazılarak düzeltildi.

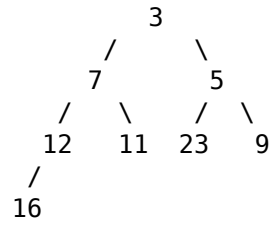
Hoca son olarak heap olmayan bir ağaç çizdi: bir düğümün bir tarafında ondan küçük, diğer tarafında ondan büyük bir değer vardı (12 örneği). Bu ağaç ne min heap ne max heap'tir.

Özetle, bir heap ağacı dengelidir, sola dayalıdır ve ya max heap ya da min heap özelliğini taşır. Hangisinin kullanılacağı uygulamanın ihtiyacına göre belirlenir.

5. Heap'in Dizide Saklanması

Sıradan bir ikili ağaç (ya da BST) dizide saklandığında, i. gözdeki düğümün sol çocuğu $2i+1$., sağ çocuğu $2i+2$. gözde tutulur (i sıfırdan başlar). Bu, ağaç dengeli değilse çok boş göz oluşmasına neden olur. Hocanın örneğinde ağaç sürekli sağa doğru gidiyordu: 2. gözdeki düğümün sağ çocuğu $2 \cdot 2 + 2 = 6$. gözde, onun sağ çocuğu $2 \cdot 6 + 2 = 14$. gözdedir. Aradaki gözlerin çoğu boş kalır.

Heap ağacında bu sorun yoktur. Hem dengeli hem sola dayalı olduğu için arada boş göz kalmaz. Hocanın örnek min heap'i:



Gözler 0'dan 7'ye kadar numaralanırsa dizi şöyle olur:

İndis	0	1	2	3	4	5	6	7
Değer	3	7	5	12	11	23	9	16

Bu yüzden heap ağaçları dizilerde saklanır. Elemanlara yine i, $2i + 1$, $2i + 2$ kuralıyla erişilir.

> **Hocanın vurgusu:** Heap ağacı dengeli ve sola dayalı olduğu için dizide arada boş göz kalmaz; bu yüzden heap'ler dizide saklanır.

6. Heap'e Ekleme ve Sift Up

6.1 Heap'in eleman eleman oluşturulması (min heap)

Bir heap ağacı şöyle oluşturulur: gelen eleman önce ilk boş yere, yani dizinin son gözüne yerleştirilir. Sonra ebeveyniyle karşılaştırılır. Min heap'te ebeveyn daha büyükse yer değiştirirler. Bu işlem eleman köke ulaşana ya da ebeveyni kendinden küçük olana kadar sürer.

Hocanın örneğinde sırasıyla 20, 15, 23, 8, 12, 11 eklendi:

Gelen	İşlem	Dizi (sonuç)
20	İlk eleman, köke yerleşir	20
15	Son göze konur; 15 < 20, yer değiştir	15, 20
23	23 > 15, yerinde kalır	15, 20, 23
8	8 < 20, yer değiştir; 8 < 15, yer değiştir, köke çıkar	8, 15, 23, 20
12	12 < 15, yer değiştir; 12 > 8, dur	8, 12, 23, 20, 15
11	Dizi önce 8, 12, 23, 20, 15, 11 olur; 11 < 23, yer değiştir; 11 > 8, dur	8, 12, 11, 20, 15, 23

Bu sonuçlar derlenen C koduyla birebir doğrulandı.

Elemanı bu şekilde yukarı doğru taşıma işlemine **sift up** denir.

> **Ders sonrası not:** Altyazıda terim "shift up / shift down" olarak geçiyor. Literatürdeki terim sift up / sift down'dır.

8 eklenirken yalnız ağacın sol kolunda ilerlendi, sağ tarafa hiç bakılmadı. Ekleme sırasında hep ağacın bir kolu etkilenir. En kötü durumda eklenen eleman köke kadar çıkar. Bu yüzden ekleme karmaşıklığı $O(\log n)$'dir.

6.2 İkinci ekleme örneği

Oluşmuş bir min heap'e 16 eklendi. 16 önce 23 ile karşılaştırılır ve yer değiştirir; sonra 20 ile karşılaştırılır ve yine yer değiştirir. 16 köke yerleşir. Ağacın yeni hali dizi olarak 16, 28, 20, 41, 47, 35, 23'tür.

Hoca burada da ağacın bir kısmının hiç işleme girmediğini vurguladı: eleman her adımda ya hep solda ya hep sağda kalan tek bir yol üzerinde ilerler. Bu yüzden n elemanlı bir ağaçta eklemenin en kötü durum karmaşıklığı $O(\log_2 n)$'dir. Hocanın örneğiyle, 8 elemanlı bir heap'te eleman en fazla 3 adımda yerine ulaşır.

> **Ders sonrası not:** Hoca 8 eleman için "3 seviye" dedi. 8 elemanlı heap'te kök dahil 4 seviye vardır; yüksekliği $\lfloor \log_2 8 \rfloor = 3$'tür. Eleman en fazla 3 kez yukarı çıkar, yani hocanın "en fazla 3 adım" sonucu doğrudur.

6.3 Son boş göz nasıl bulunur?

Soru: Yeni elemanı sona ekliyoruz; o son noktayı nasıl buluyoruz, bir değişkende mi tutuyoruz?
Cevap: Dizideki eleman sayısı n zaten tutulur; normalde bir diziyi kaç elemanla doldurduğumuzu biliriz. Yeni eleman n. göze yazılır, ardından n bir artırılır.

Hoca cevabı bir örnekle gösterdi. Dizi `4, 7, 5, 12, 21, 13, 20` olsun; $n = 7$. 10 eklenecek.

- 10, 7. göze yazılır.
- Eleman ebeveyniyle karşılaştırılır. i. gözdeki elemanın ebeveyni $\lfloor (i-1)/2 \rfloor$. gözdedir. Hoca bunu sınadı: $i = 7$ için ebeveyn $(7-1)/2 = 3$; $i = 4$ için ebeveyn $(4-1)/2 = 1$ (tam sayı bölmesi).
- 3. gözde 12 vardır. 10 < 12, yer değiştirir. 10 artık 3. gözdedir.
- Yeni ebeveyn $(3-1)/2 = 1$. gözdeki 7'dir. 10 > 7 olduğu için durulur.
- Sonuç: `4, 7, 5, 10, 21, 13, 20, 12`; n artık 8'dir.

Bu soru derste sözlü olarak öğrencilere de soruldu: "Eleman i. gözde ise ebeveyni kaçıncı gözdedir?" Bir öğrenci $(i-1)/2$ cevabını verdi.

6.4 Ekleme ve sift up kodu (min heap)

Min heap ve max heap için kod farklıdır; burada min heap yazıldı. `insert` fonksiyonu eklenecek elemanı (`item`), heap dizisini ve eleman sayısını alır. n fonksiyon içinde artırılacağı için pointer olarak geçirilir.

Adımlar:

1. `item`, heap dizisinin n. gözüne yazılır.
2. Sift up yapılır: eleman uygun yere gelene kadar yukarı taşınır.
3. n bir artırılır.

Hoca önce küçük bir örnek verdi: dizi 5, 7, 11 ($n = 3$), gelen eleman 6. 6, 3. göze yazılır. Ebeveyni $(3-1)/2 = 1$. gözdeki 7'dir; 7 > 6 olduğu için yer değiştirirler. 6'nın yeni adresi 1 olur. Ebeveyni 0. gözdeki 5'tir ve 5 < 6 olduğu için durulur. Sonuç 5, 6, 11, 7'dir (ara adım eklendi; hoca bu örnekte yalnız 6 ile 7'nin yer değiştirmesini gösterdi).

`siftUp` fonksiyonu bir şey döndürmez. Eklenen eleman, heap dizisi ve elemanın bulunduğu adres parametre olarak verilir. Hoca adresin pointer olmasına gerek olmadığını, değer olarak geçirilebileceğini söyledi. Döngü iki şart sağlandığı sürece sürer: adres 0'dan büyük olmalı ve durma bayrağı (`stop`) kurulmamış olmalı. Her adımda ebeveyn adresi `(address - 1) / 2` ile hesaplanır. Ebeveyndeki değer elemandan büyükse (min heap) yer değiştirilir ve adres ebeveynin adresi olur. Değilse `stop = 1` yapılır.

```
void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

void siftUp(int item, int heap[], int address) {
    int parent, stop = 0;
    while (address > 0 && !stop) {
        parent = (address - 1) / 2;
        if (heap[parent] > item) {          /* min heap */
            swap(&heap[address], &heap[parent]);
```

```
            address = parent;
        } else {
            stop = 1;
        }
    }
}

void insert(int item, int heap[], int *n) {
    heap[*n] = item;
    siftUp(item, heap, *n);
    (*n)++;
}
```

Hoca kodu yazarken `parent` değişkenini tanımlamayı unuttu.

Soru: Hocam, `parent`'ı tanımlamadık. **Cevap:** Doğru; `parent` tanımlandı ve `stop` sıfır olarak başlatıldı.

6.5 Sift up kodunun izlenmesi

Başlangıç dizisi `5, 11, 7, 13, 12, 16`, $n = 6$. Hoca önce 8 eklemeyi düşündü, sonra köke kadar çıkan bir örnek görülsün diye eklenen değeri 3 yaptı.

Adım	`address`	`parent`	Karşılaştırma	İşlem	Dizi
Başlangıç	6	-	-	3, 6. göze yazılır	5, 11, 7, 13, 12, 16, 3
1	6	2	7 > 3	yer değiştir, `address = 2`	5, 11, 3, 13, 12, 16, 7
2	2	0	5 > 3	yer değiştir, `address = 0`	3, 11, 5, 13, 12, 16, 7
3	0	-	`address > 0` sağlanmaz	döngü biter	3, 11, 5, 13, 12, 16, 7

Sonuç dizisi `3, 11, 5, 13, 12, 16, 7`'dir. C koduyla aynı sonuç elde edildi.

Hoca adresin 0 olmasını açıklarken "ağaç boştu, silme yapmaya çalışıyorsunuz" gibi bir ifade kullandı.

> **Ders sonrası not:** `address == 0` olması, elemanın köke ulaştığı ve karşılaştırılacak bir ebeveyn kalmadığı anlamına gelir. Ağacın boş olmasıyla ilgisi yoktur; döngünün durma şartı budur.

6.6 Heapify

Yeni eleman sona konduğunda heap özelliği bozulabilir. Örneğin hocanın ağacında 8 eklendiğinde $8 < 11$ olduğu için özellik bozulur ve 8'in yukarı taşınması gerekir. Heap özelliğini yeniden sağlamak için yapılan bu işlemlere **heapify** denir. Yukarı doğru düzenleme sift up, aşağı doğru düzenleme sift down'dır; ikisi de birer heapify işlemidir. Eklemenin karmaşıklığı $O(\log n)$'dir.

7. Heap Sort

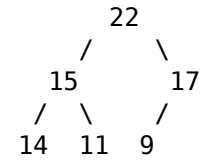
Heap üzerinde yapılan en temel işlemlerden biri, heap ağacını kullanarak sıralama yapmaktır: heap sort.

7.1 Temel fikir

Hoca önce kökü 7 olan bir min heap çizdi ve sordu: "Burada kesin olarak bildiğimiz şey ne?" Cevap: 7 bu dizinin en küçük elemanıdır. Kökte her zaman, min heap ise en küçük, max heap ise en büyük eleman bulunur. Bu özellik kullanılarak dizi sıralanabilir.

7.2 Max heap ile sıralama örneği

Hoca bu kez değişiklik olsun diye bir max heap aldı. Dizi: `22, 15, 17, 14, 11, 9`.



Hoca diziyi önce `22, 15, 7, ...` diye yazdı; bir öğrenci 17 yerine 7 yazıldığını fark etti. Hoca 7 bırakılırsa 9 aşağı inmeyeceğini ve örneğin anlamsızlaşacağını söyleyerek değeri 17'ye düzeltti.

Adım 1: Kök 22 en büyük elemandır. 22 ile son eleman 9 yer değiştirir. 22, dizi sıralıyken olması gereken yere yerleşmiş olur; bundan sonra o göze bir daha bakılmaz. Bakılan eleman sayısı $n - 1$'e iner.

9 kökte heap özelliğini bozar ve aşağı taşınmalıdır. Hoca burada öğrencilere "9'u aşağı nasıl taşırsınız?" diye sordu. Cevabı kendisi verdi: iki seçenek vardır, ya sol ya sağ çocukla yer değiştirmek. Yukarıya en büyük elemanın gelmesi gerektiğine göre önce iki çocuk (15 ve 17) birbiriyle karşılaştırılır. Büyük olan 17 ile 9 karşılaştırılır. $17 > 9$ olduğu için yer değiştirirler. 9'un altında başka eleman kalmadığı için (22'ye artık bakılmıyor) durulur.

Neden önce iki çocuk karşılaştırılır? 15 yukarı alınsaydı bu kez 15 ile 17 karşılaştırılıp 17'nin yukarı alınması gerekecek, iş uzayacaktı. Büyük çocuk yukarı alınırsa, yukarı çıkan değer diğer çocuktan da büyük olduğu için heap özelliği tek seferde sağlanır.

Durum: `17, 15, 9, 14, 11 | 22`.

Adım 2: 17 ile bakılan bölgenin son elemanı 11 yer değiştirir. 17 de yerleşir; ikinci en büyük eleman olarak 22'nin hemen önüne gelir. 11 kökte, çocukları 15 ve 9'dur. 15 büyüktür, $15 > 11$ olduğu için yer değiştirirler.

Hoca bu adımın sonucunu `15, 11, 9, 14 | 17, 22` olarak verdi.

> **Ders sonrası not:** Bu adımda sift down erken bırakıldı. 11, 1. göze indikten sonra tek çocuğu 3. gözdeki 14'tür ve $14 > 11$ olduğu için 11 bir kez daha inmelidir. Doğru durum `15, 14, 9, 11 | 17, 22`'dir. Hocanın verdiği `15, 11, 9, 14` bir max heap değildir (python ile kontrol edildi). Bir sonraki adımın sonucu (`14, 11, 9 | 15, 17, 22`) her iki yoldan da aynı çıktığı için sıralamanın sonucu değişmedi.

Adım 3: Hoca 15 ile 14'ü yer değiştirdi ve `14, 11, 9, 15, 17, 22` elde etti. 14 kökteyken 11 ile 9 karşılaştırılır; 11 büyüktür. 11 ile 14 karşılaştırılır; 14 büyük olduğu için 14 yerinde kalır. Artık 15, 17, 22 yerleşmiştir. (Doğru ara durumdan gidilirse 15 ile 11 yer değiştirir, 11 kökten iner ve yine `14, 11, 9 | 15, 17, 22` elde edilir.)

Adım 4: 14 ile 9 yer değiştirir: `9, 11 | 14, 15, 17, 22`. 9'un tek çocuğu 11'dir ve 11 büyüktür; yer değiştirirler: `11, 9 | 14, 15, 17, 22`.

Adım 5: 11 ile 9 yer değiştirir. Tek eleman kaldığı için iş biter.

Sonuç: `9, 11, 14, 15, 17, 22`. Dizi artan sırada sıralanmıştır.

Derlenen kodun her adımdaki çıktısı:

i	Dizi
5	17, 15, 9, 14, 11, 22
4	15, 14, 9, 11, 17, 22
3	14, 11, 9, 15, 17, 22
2	11, 9, 14, 15, 17, 22
1	9, 11, 14, 15, 17, 22

Hoca her adımda bakılan bölgenin bir küçüldüğünü vurguladı: başta $n = 6$, sonra 5, sonra 4.

8. Sift Down ve findBigChild Fonksiyonu

8.1 heapSort döngüsü

Hoca, işlemler kısa olduğu için akış diyagramı yerine doğrudan kod yazdığını söyledi. Yapılan iş, her seferinde bir eleman azaltarak şudur:

- Kökteki eleman ile `heap[i]` yer değiştirilir. Önce $n-1$. göz, bir sonraki adımda $n-2$. göz kökle değiştirilir; bir göz yerleştikten sonra ona bir daha bakılmaz.
- Sonra köke gelen eleman sift down ile aşağı taşınır. Sift down'a i verilir; i, bu adımda bakılacak bölgenin eleman sayısıdır (0'dan $i-1$'e kadar olan gözler).

Hoca bunu `15, 8, 12, 4, 7` dizisiyle gösterdi: önce 15 ile 7 yer değişir. $i=4$'ken bir sonraki adımda $i=3$ olur ve 12 ile 4 yer değiştirir.

8.2 siftDown fonksiyonu

Sift down'da `parent` 0'dan başlar. Köke gelen elemanın hangi çocukla karşılaştırılacağına karar vermek için **büyük çocuğun adresini** bulan ayrı bir fonksiyon (`findBigChild`) kullanılır. Karşılaştırılacak çocuk kalmadıysa bu fonksiyon -1 döndürür. Döngü, çocuk adresi -1 olmadığı ve çocuktaki değer ebeveyndekinden büyük olduğu sürece sürer. Her adımda çocuk ile ebeveyn yer değiştirir, ebeveyn adresi çocuk adresi olur ve yeni büyük çocuk bulunur.

```
void siftDown(int heap[], int n) {
    int parent = 0;
    int childAddr = findBigChild(heap, parent, n);
    while (childAddr != -1 && heap[childAddr] > heap[parent]) {
        swap(&heap[childAddr], &heap[parent]);
        parent = childAddr;
        childAddr = findBigChild(heap, parent, n);
    }
}

void heapSort(int heap[], int n) {
    for (int i = n - 1; i > 0; i--) {
        swap(&heap[0], &heap[i]);   /* en büyük eleman bölgenin sonuna */
        siftDown(heap, i);          /* yalnız ilk i elemana bakılır */
    }
}
```

Hoca kodu yazarken döngü koşulunu önce "küçüktür" diye söyledi, sonra "büyüktür" olarak düzeltti: çocuk ebeveynden büyük olduğu sürece (max heap) aşağı inilir.

8.3 Sift down örnekleri

Örnek 1: Hoca `28, 14, 11, 10, 12` max heap'iyle başladı. 28 ile 12 yer değiştirdi; bakılan bölge 4 elemandır ($i=4$): `12, 14, 11, 10`. 12'nin çocukları 14 ve 11'dir; büyük olan 14'tür ve 12 onunla karşılaştırılır. Hoca bu noktada `findBigChild` fikrine geçti ve örneği tamamlamadı.

Örnek 2: Hoca daha fazla hareket görülsün diye ağacı büyüttü ve bazı değerleri değiştirdi (4'ün bir yerde inmeyi bırakması görülsün diye 15'in çocuklarını 3 ve 2 yaptı). 28 ile 4 yer değiştirdikten sonra bakılan bölge `4, 11, 15, 9, 7, 3, 2` biçimindedir; 28 sonda yerleşmiştir.

Adım	`parent`	Çocuklar	Büyük çocuk adresi	Karşılaştırma	Dizi
1	0	11, 15	2	15 > 4, yer değiştir	15, 11, 4, 9, 7, 3, 2
2	2	3, 2	5	3 > 4 değil, dur	15, 11, 4, 9, 7, 3, 2

Hocanın vurguladığı nokta, 4'ün 15 ile yer değiştirdikten sonra çocuklarının (3 ve 2) büyüğünden bile büyük olduğu için yerinde kalmasıdır.

Örnek 3: Hoca öğrencilere bir örnek daha isteyip istemediklerini sordu ve bir örnek daha yaptı. Max heap `15, 9, 7, 8, 4, 5` iken 15 ile 5 yer değiştirdi. Bakılan bölge `5, 9, 7, 8, 4` (0-4. gözler), 15 yerleşmiştir.

Adım	`parent`	Çocuklar	Büyük çocuk adresi	İşlem	Dizi
1	0	9, 7	1	9 > 5, yer değiştir	9, 5, 7, 8, 4
2	1	8, 4	3	8 > 5, yer değiştir	9, 8, 7, 5, 4
3	3	7. ve 8. gözler bölge dışında	-1	döngü biter	9, 8, 7, 5, 4

3. gözdeki elemanın çocukları $2 \cdot 3 + 1 = 7$. ve $2 \cdot 3 + 2 = 8$. gözlerde olabilir. Bunlar bakılan bölgenin dışında olduğu için `findBigChild` -1 döndürür. Heapify sonunda dizi `9, 8, 7, 5, 4, 15` olur.

   Ders sonrası not: Altyazıda sonuç bir yerde "9, 8, 2, 5, 4, 15" olarak geçiyor; hoca hemen ardından "9, 8, 7, 5, 4, 15" diye düzeltti. Doğrusu 9, 8, 7, 5, 4, 15'tir (kodla doğrulandı).

Bu örnekten sonra bir öğrenci ağacın dengeli olup olmadığını sordu.

Soru: Bu şekilde heap geçerli mi? Heap'lerin dengeli olması gerektiğini söylemiştik; 7'nin iki çocuğu olması gerekmez mi? **Cevap:** Hoca önce sıralama sırasında ağaca yalnız bakılan bölgeye kadar bakıldığını, kökten inen eleman yüzünden zaten heap özelliğinin geçici olarak bozulduğunu ve ağacın sürekli küçüldüğünü anlattı. Öğrenci sorusunu açtı: "5'in indisi 7 olması gerekmiyor mu? Çünkü $2i+1$ diyorsunuz." Hoca bu kez öğrencinin haklı olduğunu kabul etti: çizimde ağacın dengesi bozulmuştu, çünkü sol tarafa bakarken öbür tarafı atlamıştı. 7'nin altına iki eleman ekleyerek çizimi düzeltti.

8.4 findBigChild: büyük çocuğu bulmak

Hoca yeni bir küçük örnekle başladı: `17, 11, 13, 8` gibi bir ağaçta 17 ile 8 yer değiştirdikten sonra heapify için 11 ve 13'ten büyük olanın 8 ile değiştirilmesi gerekir. Büyük olan (13) yukarı taşınırsa 11'den de büyük olacağı için heap özelliği tek seferde yeniden sağlanır. Bu yüzden köke gelen elemanın kimle karşılaştırılacağına önce karar verilmelidir.

Üç durum vardır:

1. **Ebeveynin iki çocuğu var.** Hocanın örneğinde ebeveyn 4, çocuklar 15 ve 17'dir; 4, büyük olanla karşılaştırılır. Ebeveyn p. gözde ise çocuklar $2p+1$. ve $2p+2$. gözlerdedir. İki çocuk olması için eleman sayısı $n > 2p+2$ olmalıdır. Kök için ($p = 0$) bu, $n > 2$, yani $n \geq 3$ demektir. Büyük çocuğun adresi döndürülür.
2. **Ebeveynin tek çocuğu var.** $2p+1 < n$ ama $2p+2 \geq n$. Hocanın örneğinde ebeveyn 4. gözdedir, tek çocuk 9. gözdedir ve $n = 10$'dur. Doğrudan bu çocuğun adresi ($2p+1$) döndürülür.
3. **Ebeveyn yapraktır.** Örneğin 4 dokuzuncu gözdedir ve altında kimse yoktur. Karşılaştırılacak çocuk yoktur; -1 döndürülür. `siftDown` döngüsü çocuk adresi -1 olduğunda durur.

Hoca ikinci kontrolü kodlarken bir örnek daha verdi: ebeveyn 5. gözde (değeri 9), sol çocuğu 11. gözde (14), 12. göz boştur. Bu durumda $n = 12$'dir ve $2p+1 = 11 < 12$ olduğu için tek çocuk vardır.

```
int findBigChild(int heap[], int parent, int n) {
    if (n > 2 * parent + 2) {                       /* iki çocuk */
        if (heap[2 * parent + 1] > heap[2 * parent + 2])
            return 2 * parent + 1;
        else
            return 2 * parent + 2;
```

```
    } else if (2 * parent + 1 < n) {                /* tek çocuk */
        return 2 * parent + 1;
    } else {
        return -1;                                  /* ebeveyn yaprak */
    }
}
```

8.5 findBigChild ile izleme

Hoca bir örnekte 15'i sona taşıdı: bakılan bölge `8, 11, 9, 7` (0-3. gözler, $n = 4$), 15 sonda yerleşmiş durumdadır.

Adım	`parent`	Kontrol	Döndürülen	Karşılaştırma	Dizi
1	0	$4 > 2$: iki çocuk (11, 9)	1	11 > 8, yer değiştir	11, 8, 9, 7
2	1	$4 > 4$ değil; $3 < 4$: tek çocuk	3	7 > 8 değil, dur	11, 8, 9, 7

Son durumda bölge heapify edilmiştir; 15 sona taşınmış olarak durur: `11, 8, 9, 7, 15`.

9. Heap Sort Kodunun Tam İzlemesi

Hoca son olarak kodun tamamını `15, 8, 12, 4, 7` dizisi üzerinde izledi (0-4. gözler). $i, n-1 = 4$'ten başlar.

i	Swap	Sift down	Dizi
4	15 ile 7	Bölge 7, 8, 12, 4. Büyük çocuk 12; 12 > 7, yer değiştir. 7'nin (2. göz) bölgede çocuğu yok, dur	12, 8, 7, 4, 15
3	12 ile 4	Bölge 4, 8, 7. Büyük çocuk 8; 8 > 4, yer değiştir. 4'ün çocuğu yok	8, 4, 7, 12, 15
2	8 ile 7	Bölge 7, 4. Tek çocuk 4; 4 > 7 değil, dur	7, 4, 8, 12, 15
1	7 ile 4	Bölge tek eleman	4, 7, 8, 12, 15

Döngü $i > 0$ olduğu sürece sürer. $i = 0$ olduğunda bakılacak bir şey kalmaz; yalnız kök kalmıştır ve iş biter. Dizi artan sırada sıralanmıştır: `4, 7, 8, 12, 15`. Derlenen kod her adımda aynı diziyi verdi.

> **Ek bilgi (derste anlatılmadı):** Derste heap sort hep hazır bir max heap ile başladı. Sırasız bir diziyle başlanırsa önce diziden heap kurulmalıdır (build heap). Bu, elemanları tek tek ekleyerek $O(n \log n)$'de ya da son iç düğümden köke doğru her düğüme sift down uygulayarak $O(n)$'de yapılabilir. Sıralama döngüsü $n-1$ kez sift down yaptığı için heap sort'un toplam karmaşıklığı $O(n \log n)$'dir.

Örnek Sorular

Bu derste ekran karesi kullanılmadı; aşağıdaki sorular derste sözlü olarak sorulan sorulardır.

Soru 1 (derste sözlü, 00:12:26): Özyinelemeli `insert` fonksiyonunda `root == NULL` olması ne demektir?

Çözüm: Ağaç (ya da o anda bakılan alt ağaç) boştur. Yeni düğüm oluşturulur ve adresi döndürülür. Cevabı derste bir öğrenci verdi, hoca onayladı.

Soru 2 (derste sözlü, 00:22:47): Kökü 50 olan ağaçta (50; 40, 70; 60, 100) kök silinecek. Sağ alt ağacın en küçük değeri kaçtır ve silmeden sonra ağaç nasıl olur?

Çözüm: Sağ alt ağaç 70, 60, 100'dür; en küçük değeri 60'tır. 60 kökün yerine geçer. Yeni ağaç: kök 60, solunda 40, sağında 70, 70'in sağında 100.

Soru 3 (derste sözlü, 00:39:08): Min heap örneğinde 13'ün altına 11 yazılmıştır. Bu ağaç min heap midir?

Çözüm: Değildir; 11 < 13 olduğu için çocuk ebeveyninden küçüktür ve min heap şartı bozulur. Derste değer 14 yapılarak düzeltildi.

Soru 4 (derste sözlü, 01:02:51): Heap'e eklemede yeni eleman sona yazılıyor. Son boş göz nasıl bulunur?

Çözüm: Eleman sayısı n tutulur. Yeni eleman n. göze yazılır, sift up yapılır ve n bir artırılır. Örnek: `4, 7, 5, 12, 21, 13, 20` ($n = 7$) dizisine 10 eklenirse 10 önce 7. göze yazılır, 3. gözdeki 12 ile yer değiştirir, 1. gözdeki 7'den büyük olduğu için durur. Sonuç `4, 7, 5, 10, 21, 13, 20, 12`, $n = 8$.

Soru 5 (derste sözlü, 01:05:17): Heap dizisinde i. gözdeki elemanın ebeveyni kaçıncı gözdedir?

Çözüm: $\lfloor (i-1)/2 \rfloor$. gözdedir. Sağlama: $i = 7$ için $\lfloor 6/2 \rfloor = 3$; $i = 4$ için $\lfloor 3/2 \rfloor = 1$. Cevabı derste bir öğrenci verdi.

Soru 6 (derste sözlü, 01:20:12): Kökü 7 olan bir min heap'te kesin olarak bildiğimiz şey nedir?

Çözüm: Kökteki 7, dizinin en küçük elemanıdır. Min heap'te kökte en küçük, max heap'te en büyük eleman bulunur; heap sort bu özelliği kullanır.

Soru 7 (derste sözlü, 01:23:53): Max heap `22, 15, 17, 14, 11, 9`'da 22 ile 9 yer değiştirdikten sonra 9 aşağı nasıl taşınır?

Çözüm: Önce 9'un iki çocuğu (15 ve 17) karşılaştırılır; büyük olan 17'dir. 17 > 9 olduğu için 9 ile 17 yer değiştirir. 9'un artık bakılan bölgede çocuğu yoktur. Sonuç `17, 15, 9, 14, 11 | 22`. Derste öğrenciler cevap vermedi, hoca kendisi çözdü.

Soru 8 (derste sözlü, 02:01:03): Max heap `15, 9, 7, 8, 4, 5` için yapılan çizimde 5, 8'in altına yazılmıştır. Bu ağaç dengeli ve geçerli bir heap midir?

Çözüm: Altıncı eleman 5. göze ait olmalıdır, yani $2 \cdot 2 + 1 = 5$ kuralına göre 7'nin sol çocuğu olmalıdır. 8'in altına yazılırsa 7. göze ($2 \cdot 3 + 1$) düşer ve 5. ile 6. gözler boş kalır; ağaç sola dayalı ve dengeli olmaz. Derste hoca öğrencinin haklı olduğunu kabul edip çizimi düzeltti.

Dersin Toparlaması

Ders, ikili arama ağacının tekrarıyla başladı. Dengeli bir BST'de arama, ekleme ve silmenin $O(\log n)$ olduğu, ağaç tek tarafa yığılırsa bu avantajın kaybolduğu hatırlatıldı. Özyinelemeli ekleme kodu bir örnek olarak gösterildi. Ardından BST'den silmenin üç durumu ele alındı: yaprak doğrudan silinir, tek çocuklu düğümün yerine çocuğu geçer, iki çocuklu düğümün yerine sağ alt ağacın en küçük elemanı konur. Silme kodu ödeve bırakıldı.

Dersin ikinci yarısı heap ağacına ayrıldı. Heap dengeli, sola dayalı ve max ya da min heap özelliğini taşıyan bir ağaçtır. Arada boş göz kalmadığı için dizide saklanır; i. gözün çocukları $2i + 1$ ve $2i + 2$, ebeveyni $\lfloor (i-1)/2 \rfloor$. gözdedir. Eklemede eleman sona yazılıp sift up ile yukarı taşınır ($O(\log n)$). Heap sort'ta kök ile bölgenin son elemanı yer değiştirir ve köke gelen eleman `findBigChild` yardımıyla sift down ile aşağı indirilir. Hoca, heap ağacının ilerleyen haftalarda görülecek öncelikli kuyruklarda da kullanıldığını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.